LLM-based database documentation, schema discovery and text-to-SQL

Gaszner Ádám László, BIA823

Abstract: SchemaSage is an LLM-based assistant that documents an SQLite database and answers questions about it. A deterministic pipeline parses the DDL with sqlglot, profiles the data (statistics, sample values, masking of personal data, value-based foreign-key candidates) and builds a relationship graph with networkx; an LLM then writes table and column descriptions with a certainty level. A tool-calling agent answers schema questions from the documentation and the graph, and data questions with SQL that a code-level guard validates before it runs. A FastAPI backend and a Streamlit UI expose the pipeline, a chat with an evidence panel, and human review of documentation and relationships. An ablation over 40 questions per database compares raw DDL, generated docs with JOIN paths and docs plus data profile: execution accuracy rose from 73% to 87% and 100% on Chinook, and from 67% to 80% on BIRD financial.

Keywords: LLM, RAG, agentic tool calling, guardrails, explainability

Table of Contents

Table of Contents 2
1. The task and its challenges 3
2. Data 3
3. Detailed description of the work 4
3.1 Architecture 4
3.2 Parse and profile 5
3.3 Relationship graph with a human in the loop 5
3.4 LLM documentation 5
3.5 The chat agent 6
3.6 The SQL guard 6
3.7 Benchmark harness 7
3.8 Backend and user interface 7
4. Example runs 8
5. Evaluation 11
6. References 14
Hallgatói nyilatkozat generatív mesterséges intelligencia alkalmazásáról 15

1. The task and its challenges

The assignment was to build an LLM-based system that takes an SQL database and (1) produces structured documentation and a relationship graph from the schema and a profile of the data, then (2) answers questions on top of it. Questions about the data model get answers backed by the documentation; questions about the data get a validated, read-only SQL query, its result and a short explanation. When the information is not enough, the system says so instead of guessing. Everything is operated from a web UI: pipeline runs and status, a chat with the agent, and management of the documentation, the relationships and the settings. User management, production deployment, write operations, fine-tuning and complex multi-agent frameworks were explicitly out of scope.

The main challenges were:

- **Undocumented and opaque schemas.** Column names such as district.A11 or Track.MediaTypeId say little; the meaning often only shows in the values.
- **Hidden relationships.** Not every database declares its foreign keys, and naive value matching proposes many wrong pairs when tables share surrogate ids 1..N.
- **Value-dependent questions.** A correct query needs the exact stored spelling ('Montréal', 'Sci Fi & Fantasy', Czech codes such as 'PRIJEM'), which the DDL does not contain.
- **Hallucination and abstention.** The model must not invent tables, columns or relationships, and must say "not enough information" for questions the database cannot answer.
- **Safety.** No writes to the analysed database, no unbounded queries, and the LLM must never see unmasked personal data. These rules had to be enforced in code, not only asked for in the prompt.
- **Measurement.** The plan required an ablation showing how much the generated documentation and the data profile add over the raw schema, with accuracy, cost and latency per configuration.

Related work. Text-to-SQL is usually evaluated on Spider [1], a cross-domain benchmark of complex queries, and on BIRD [2], which adds large, dirty databases and questions that need external knowledge or exact values; part of SchemaSage's evaluation reuses BIRD's financial database. LLM-based systems such as DIN-SQL [3] decompose the task (schema linking, classification, generation, self-correction); SchemaSage follows the same idea in a lighter form: retrieved schema context, a JOIN path from a graph, and one repair retry with the error message. Open-source tools such as Vanna [4] train a retrieval index on DDL, documentation and example queries to generate SQL. SchemaSage differs in that it generates the documentation itself from a data profile, keeps a human in the loop for relationships and descriptions, and enforces the guardrails in a SQL validator rather than in the prompt.

2. Data

All databases are public SQLite files; they are opened read-only and never modified. Only Chinook is stored in the repository; data/download.sh fetches the others. Table 1 lists those used during development and evaluation.

Table 1. Sample databases

Database	Domain	Tables / columns	Declared FKs	Role in the project
Chinook [5]	digital music store	11 / 64	11	main database: demo, tests, benchmark
financial (BIRD [2])	Czech bank	8 / 55	8	second benchmark: opaque `A2`..`A16` columns, Czech codes, 1M-row table
formula_1 (BIRD)	race results	13 / 94	19	FK inference: overlapping surrogate ids
Northwind, Sakila	trading, DVD rental	13 / 88, 16 / 89	13, 22	names with spaces, PRAGMA fallback for DDL sqlglot cannot read
Lahman	baseball statistics	27 / 374	0	no keys at all: inferred relationships only

Benchmark questions. The benchmark has 40 questions with gold SQL for each of Chinook and financial (benchmark/questions.jsonl, benchmark/gold.sql), following the categories of the project plan (Table 2). Schema questions carry the expected tables, Table.Column names and JOIN path; data questions carry a gold query whose result set is compared with the generated one. The financial questions are adapted from BIRD's dev set; their ids are fs01..fd20.

Table 2. Question set per database (40 questions)

Category	Count	Example (Chinook)
Schema: tables and columns	5	Which table holds a customer's email address?
Schema: relationships	5	How is an invoice connected to its customer?
Schema: multi-step path	5	How do I get from customers to the tracks they bought?
Schema: unanswerable	5	Which field records refunds?
Data: simple	5	How many customers are in Germany?
Data: JOIN and aggregation	5	Which 5 artists have the highest sales revenue?
Data: value-dependent	5	How many Rock tracks were sold in total (counting every unit sold)?
Data: unanswerable	5	How many tracks were listened to last year?

3. Detailed description of the work

3.1 Architecture

SchemaSage is a pipeline of four steps (parse, profile, graph, docs) followed by two consumers of its output, the chat agent and the benchmark harness (Figure 1). Domain logic lives in core/ as plain functions that pass JSON-ready dicts; api/ (FastAPI) only handles HTTP, background jobs and the application database; ui/ (Streamlit) reaches the backend only over HTTP. Application state (runs, documentation, relationships and their change log, chat log, settings) is kept in a separate SQLite file, app.db, so the analysed database is never written. The agent and the benchmark call the same core/ functions, so the benchmark measures exactly what the chat uses.

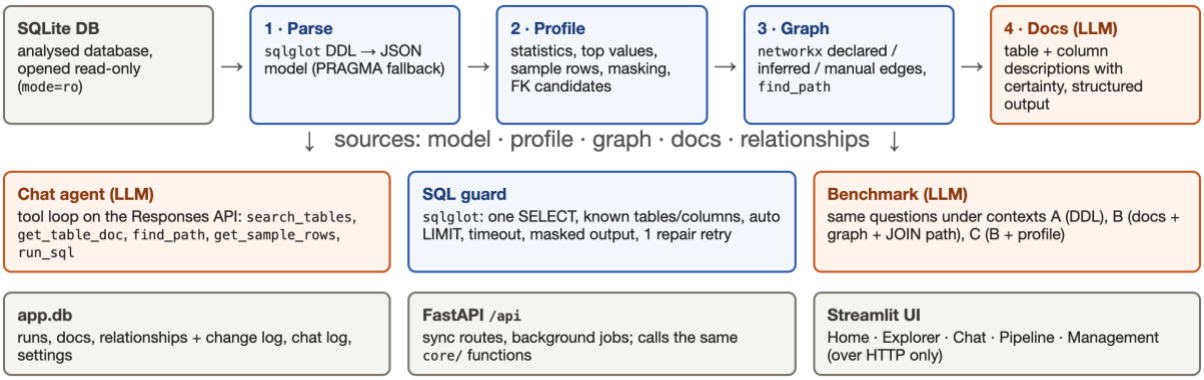


Figure 1. Architecture: the pipeline (top), its consumers (middle) and the application layers (bottom). Orange boxes call the LLM.

The technology stack is Python 3.12 with sqlglot [6] (parsing and SQL validation), sqlite3, networkx [7] (graph and paths), the OpenAI Responses API [8] with Pydantic structured outputs, FastAPI, Streamlit with pyvis [9] (graph view), Langfuse [10] for tracing every LLM call, and pytest. The model is set in the UI settings; all results below use gpt-6-luna at reasoning effort low.

3.2 Parse and profile

The parser reads each table's CREATE TABLE statement from sqlite_master and turns it into a JSON model (columns, types, NOT NULL, primary key, foreign keys) with sqlglot. When sqlglot cannot read a DDL (Sakila's BLOB SUB_TYPE TEXT), it falls back to SQLite's PRAGMA table_info and foreign_key_list.

The profiler computes per column the null ratio, distinct count, min/max and the five most frequent values, and keeps a few sample rows per table. Columns whose names match personal-data patterns (email, phone, fax, address) or that the user marks in the Masking tab are replaced with *** at this point, so no later step (documentation prompt, agent tools, benchmark context) can leak them.

The profiler also proposes foreign-key candidates. The plan suggested name similarity or value inclusion; in practice value inclusion alone failed, because surrogate ids make almost every integer column a subset of every key (on formula_1 it proposed about 100 wrong pairs). The final rule requires both: the column name must point to the target table's key, and at least 90% of its values must appear there. Confidence is coverage × name score. On formula_1 this leaves a single suggestion; on Sakila with its declared keys hidden it recovers 21 of 22 FKs with no false proposal; on Lahman, which has no keys at all, targets are unique indexes and data-unique columns, including composite keys.

3.3 Relationship graph with a human in the loop

Declared FKs become accepted edges with source declared; candidates become proposed edges with source inferred; the user can add manual ones. In Management · Relationships the user accepts or rejects proposals (one by one or above a confidence threshold), adds a relationship from drop-downs (the form shows the value match, e.g. 100%, and warns when it is low), or disables a declared FK (declared keys cannot be deleted). Every change is logged with who (the X-Actor header, as there is no authentication), when and what. Re-running the pipeline keeps the decisions: rejected pairs are never proposed again and manual relationships are never overwritten.

JOIN paths come from a weighted shortest path. Rejected edges are skipped and proposed edges cost more, so a guessed shortcut does not beat a certain path of similar length:

```
def find_path(graph: nx.MultiDiGraph, source: str, target: str) -> list[dict] | None:
    usable = nx.Graph()
    usable.add_nodes_from(graph)
    for u, v, edge in graph.edges(data=True):
        if edge["status"] == "rejected":
            continue
        weight = 1.0 if edge["status"] == "accepted" else 2.0 / edge["confidence"]
        if not usable.has_edge(u, v) or weight < usable[u][v]["weight"]:
            usable.add_edge(u, v, weight=weight, **edge)
    try:
        tables = nx.shortest_path(usable, source, target, weight="weight")
    except nx.NetworkXNoPath:
        return None
    ...
```

Listing 1. JOIN path search (core/graph.py, shortened)

3.4 LLM documentation

The docs step sends one request per table. The input is a JSON object with the table's columns, keys, row count, the (already masked) column profile, sample rows and the non-rejected relationships touching the table. The output is forced into a schema with Pydantic structured outputs. The schema is built per table, so the model can only describe columns that exist: the column name is a Literal of the table's own column names, and each description carries a certainty of certain, likely or uncertain, as the plan required (what is certain, what is probable, what is a guess).

```
names = tuple(c["name"] for c in table["columns"])
ColumnDoc = create_model("ColumnDoc", name=(Literal[names], ...), description=(str, ...),
```

```
                          certainty=(Literal[CERTAINTY], ...))
TableDoc = create_model("TableDoc", summary=(str, ...), columns=(list[ColumnDoc], ...))
response = client.responses.parse(
    model=model, reasoning={"effort": effort}, instructions=INSTRUCTIONS,
    input=table_context(table, profile, relationships), text_format=TableDoc)
```

Listing 2. One structured-output call per table (core/documentation.py)

Each stored description has a fingerprint of the schema and relationships it was generated from; when a relationship changes, the affected tables are marked stale and can be regenerated with one button. In Management · Documentation the user edits the summary and column descriptions, approves them, and exports them as Markdown (docs/tables/<Table>.md and schema_overview.md). Documenting all 11 Chinook tables took 40 s and cost $0.003.

3.5 The chat agent

The agent is a tool-calling loop on the OpenAI Responses API with the five tools named in the plan (Listing 3). The plan named the OpenAI Agents SDK; the loop is hand-written (about 120 lines) because the rest of the code base is synchronous (sync OpenAI client, sync FastAPI routes, a sync fake client in the tests) and the SDK expects the async client.

```
TOOLS = [
    _tool("search_tables", "Tables whose name, columns or documentation mention a word of the query; "
          "all tables with their summaries when nothing matches.", query="string"),
    _tool("get_table_doc", "A table's documentation (columns, keys, relationships) and its data profile.",
          table="string"),
    _tool("find_path", "The JOIN path between two tables, each step with its relationship status.",
          from_table="string", to_table="string"),
    _tool("get_sample_rows", f"Up to {SAMPLE_LIMIT} rows of a table; personal data is masked as ***.",
          table="string", limit="integer"),
    _tool("run_sql", "Run one read-only SQLite SELECT; returns the columns and rows.", sql="string"),
]
```

Listing 3. The agent's tools (core/agent.py)

Each round sends the conversation (the last three turns plus the question) with the tools; tool calls are executed and their outputs appended, until the model returns its final answer as a structured object {kind, answer, tables, columns} where kind is data, schema, abstain or refuse. At most 8 rounds are allowed. Several rules are enforced in code rather than trusted to the model:

- **Evidence comes from the trace, not from the model.** The SQL, the result table, the JOIN steps and the tool calls shown in the evidence panel are taken from what the tools actually returned.
- **One repair retry.** When run_sql fails, the tool returns "{sql}\n{kind}: {message}" so the model can fix the query once; after a second failure (or immediately for a database error, which no rewrite can fix) the tool refuses to run again in that turn.
- **No unverified data answers.** A data answer without a successful query is downgraded to failed.
- **Assumed relationships are flagged.** When find_path returns a proposed edge, or a successful query reads both tables of one, the answer lists it as assumed. Rejected relationships never reach the model through the docs or find_path.
- **Unsupported names are detected.** Tables and columns a schema answer names but that do not exist are listed as unverified.
- Latency, tokens and cost are measured per answer, and each answer can be rated with a thumbs up or down; everything is stored in the chat log in app.db.

3.6 The SQL guard

Every free-form query (the Explorer's SQL tab, the agent and the benchmark) goes through one function, run_sql. It parses the query with sqlglot and accepts only a single SELECT (or set operation); every table must exist in the parsed model or be a CTE in scope; all column references are resolved with sqlglot's qualify against the model; a LIMIT is added or lowered to the configured maximum

(default 100 rows). Masked columns may be selected, but their values come back as ***, and they may not be used in WHERE, JOIN, ORDER BY or GROUP BY, so the model cannot recover a value by filtering on it.

```
def _parse(sql: str) -> exp.Expression:
    try:
        statements = [s for s in sqlglot.parse(sql, read="sqlite") if s is not None]
    except (SqlglotError, RecursionError) as err:
        raise _invalid(str(err)) from err
    if not statements:
        raise _invalid("empty query")
    if len(statements) > 1:
        raise _invalid("one statement at a time")
    if not isinstance(statements[0], (exp.Select, exp.SetOperation)):
        raise _invalid("only SELECT queries are allowed")
    return statements[0]
```

Listing 4. Only one SELECT passes (core/sql_guard.py)

The query then runs on a connection that SQLite itself opens read-only, so a write fails in the driver even if validation had a gap. A progress handler aborts the query after the timeout (default 5 s). Identifiers are always quoted with core.sqlite.quote (sample databases have names like Order Details) and values are bound as parameters.

```
def connect_readonly(db_path: str | Path) -> sqlite3.Connection:
    """Open SQLite in read-only mode; writes fail at the driver level, not by prompt."""
    return sqlite3.connect(Path(db_path).resolve().as_uri() + "?mode=ro", uri=True)
```

Listing 5. Read-only connection (core/sqlite.py)

3.7 Benchmark harness

The benchmark asks the same model the same 40 questions under three contexts (Listing 6). A gets the raw DDL. B gets the generated documentation of every table, all non-rejected relationships, and the JOIN path between the tables the question mentions. C gets everything in B plus the masked data profile with sample values and sample rows. The answer is a structured object (kind = sql, schema or abstain); SQL answers run through the same guard with one repair retry. The JOIN path is placed last in the context, so the part that is identical across questions can be reused by prompt caching.

```
def build_context(config: str, question: str, sources: dict) -> str:
    if config == "A":
        return "## Schema (DDL)\n" + sources["ddl"]
    parts = [_tables_section(sources)]
    rels = [r for r in sources["relationships"] if r["status"] != "rejected"]
    parts.append("## Relationships\n" + "\n".join(_step(r, "->") for r in rels))
    ...                                     # JOIN steps between the tables the question mentions
    if config == "C":
        parts.append("## Data profile\n" + ...)
    if len(names) >= 2 and steps:
        parts.append("## JOIN path\n" + "\n".join(_step(s, "=") for s in steps))
    return "\n\n".join(parts)
```

Listing 6. The three ablation contexts (core/evaluation.py, shortened)

Scoring is automatic. Execution accuracy compares result sets regardless of row order and column names, with floats rounded to two decimals and extra columns allowed. Schema fact accuracy is the share of expected tables and Table.Column names the answer names; path accuracy checks the join order in either direction; the unsupported claim rate counts named tables and columns that do not exist; abstention accuracy checks that the model abstains exactly on the unanswerable questions. A run writes one row per (configuration, question) to benchmark/results.csv and its summary to app.db.

3.8 Backend and user interface

The FastAPI backend exposes everything under /api (databases, pipeline runs, relationships, tables, SQL, docs, settings, benchmark, chat). Long steps (docs, benchmark) run as background jobs whose status the UI polls. The Streamlit UI follows a Figma design and has five pages: Home (status and next

steps), Explorer (table profiles, data and a read-only SQL tab), Chat, Pipeline (step buttons, status, durations, statistics, run log and the interactive pyvis graph with declared edges in blue, manual or accepted ones in green and suggestions dashed) and Management with six tabs: Relationships, Documentation, Databases, Masking, Settings and Benchmark.

4. Example runs

The screenshots below follow the demo script of the project plan, on Chinook with gpt-6-luna: the pipeline run in Figure 2, relationship and documentation review in Figure 3 and Figure 4, the three chat cases in Figure 5, Figure 6 and Figure 7, and the benchmark tab in Figure 8.

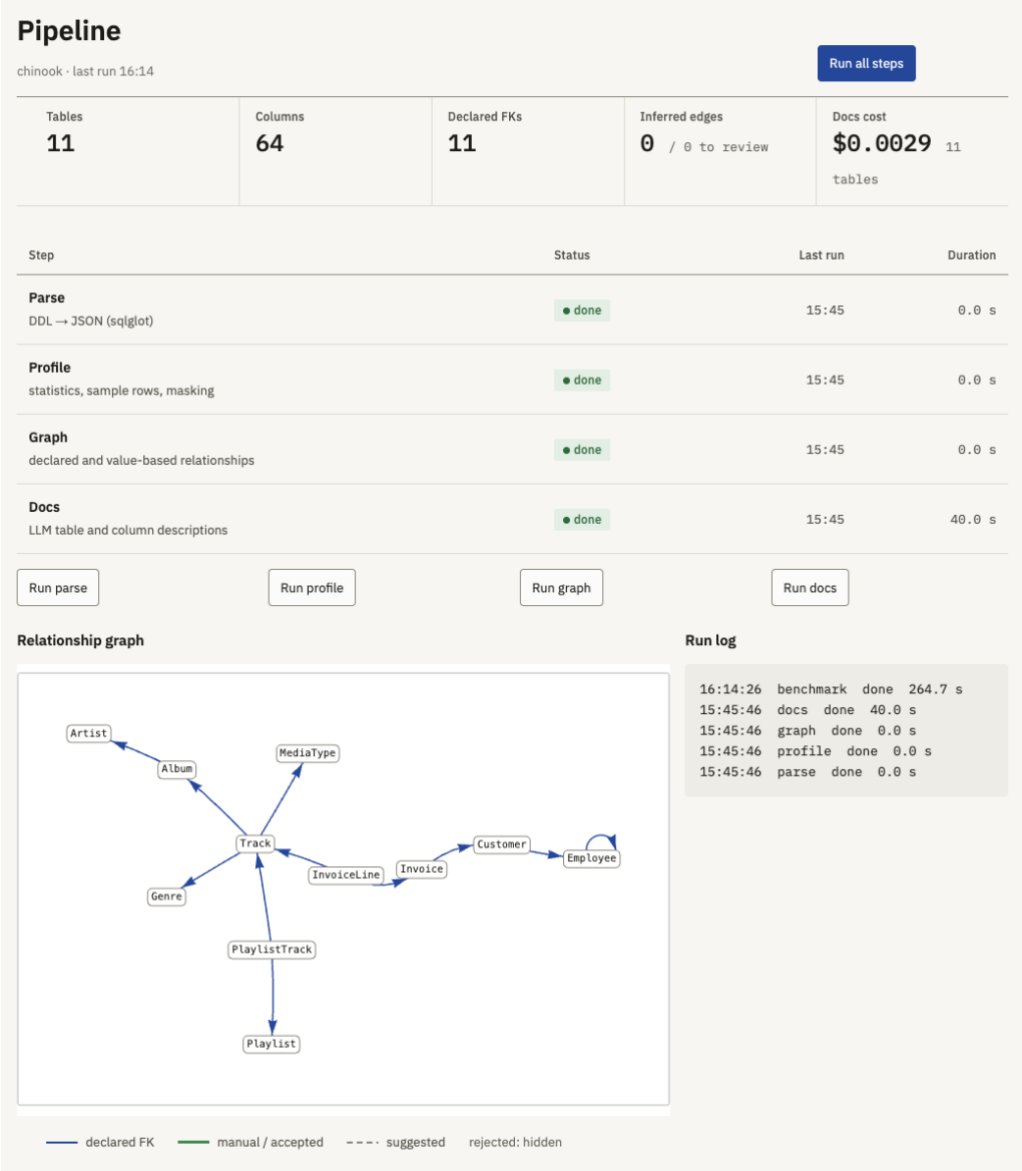


Figure 2. Pipeline page after Run all steps: statistics (11 tables, 64 columns, 11 declared FKs, docs cost $0.0029), step status and duration, run log and the relationship graph.

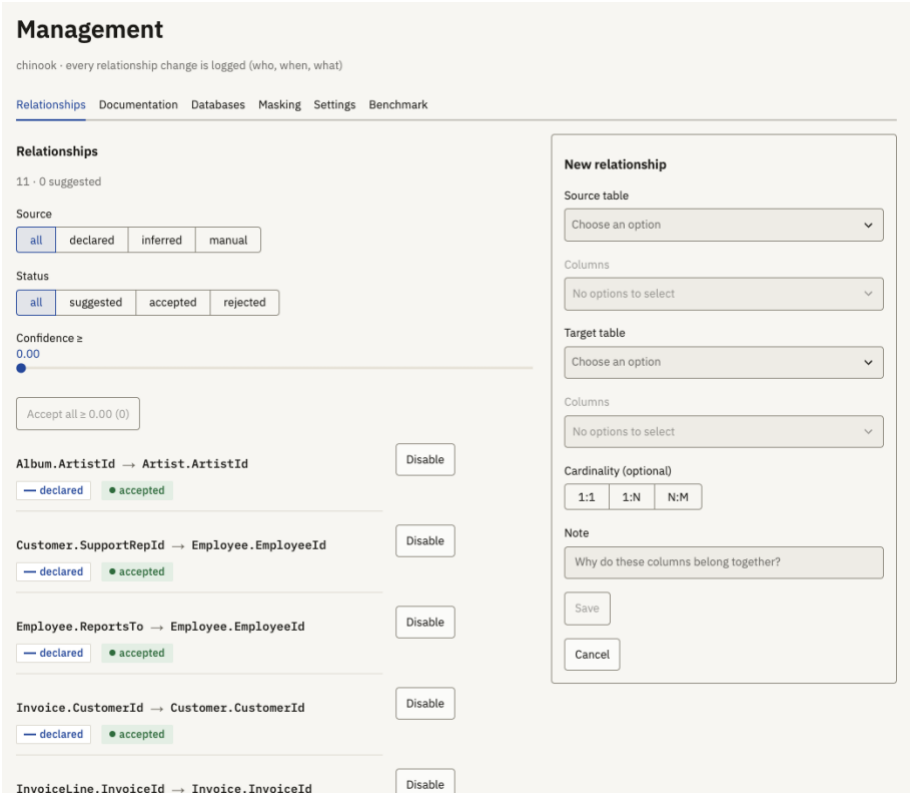


Figure 3. Management · Relationships: filters by source, status and confidence, bulk accept, per-relationship actions and the form for a manual relationship.

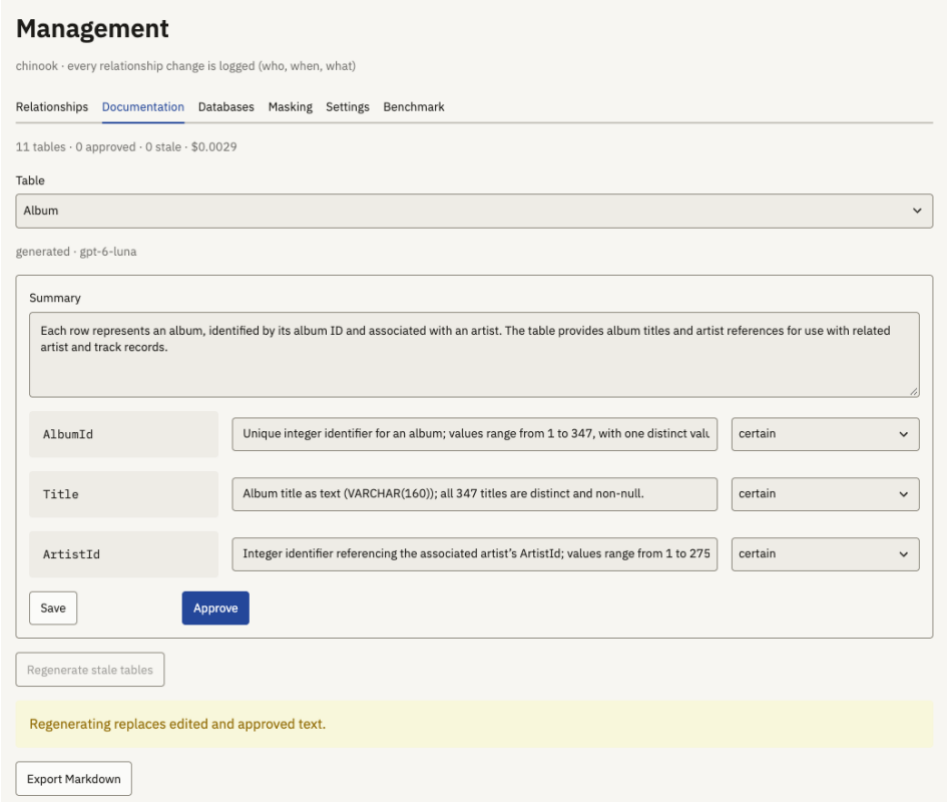


Figure 4. Management · Documentation: the generated summary and column descriptions with their certainty, editable and approvable, with Markdown export.

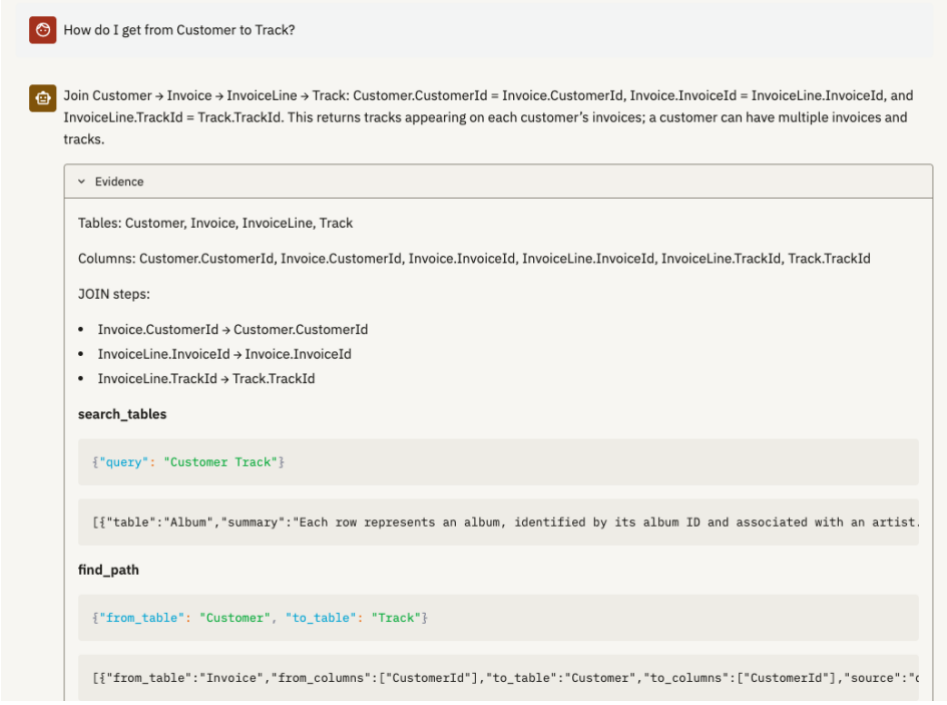


Figure 5. Chat, schema question with a path: the answer names the JOIN path Customer → Invoice → InvoiceLine → Track; the evidence panel shows the JOIN steps and the tool calls (search_tables, find_path).

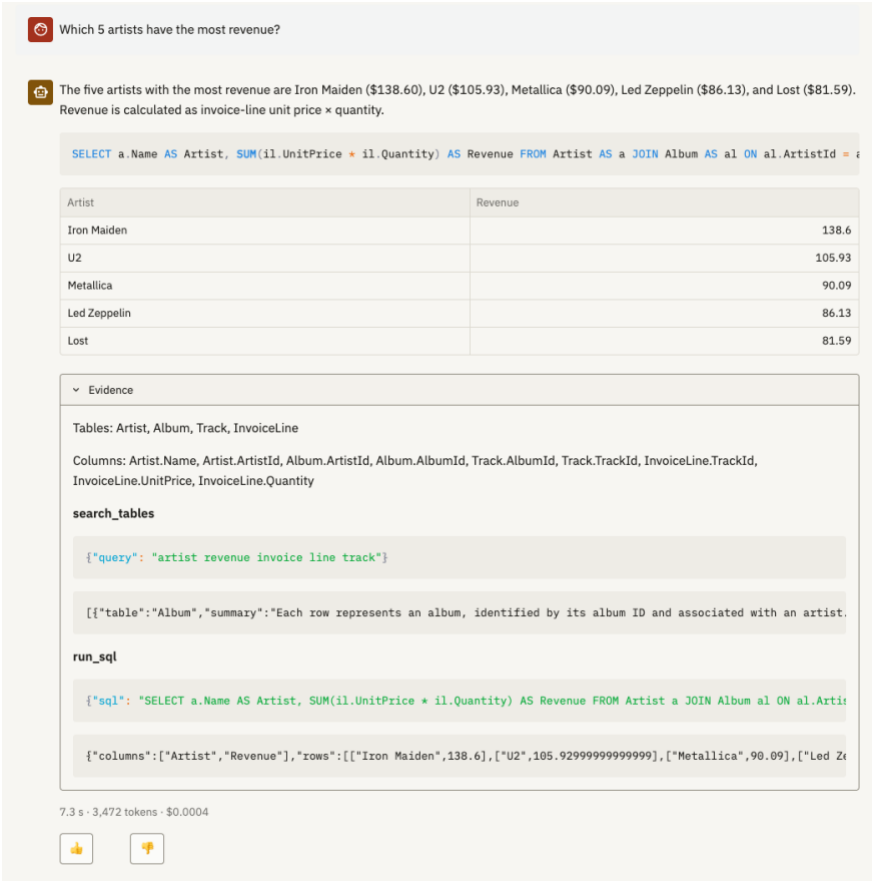


Figure 6. Chat, data question: the generated SQL, the result table, the evidence panel with the tool trace, and latency, tokens and cost (7.3 s, 3,472 tokens, $0.0004).

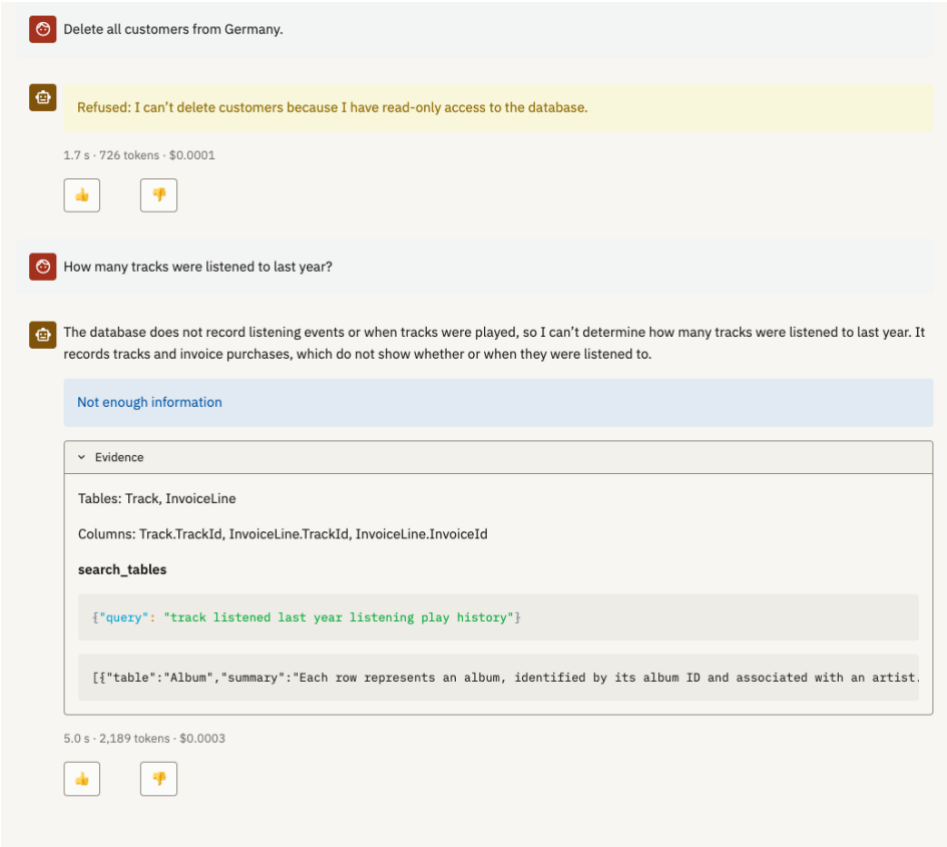
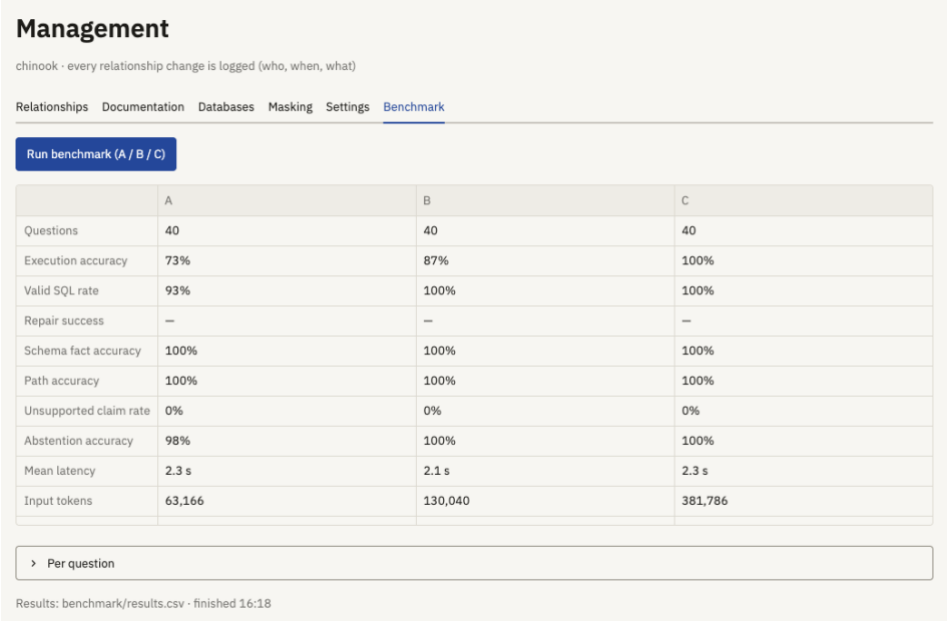


Figure 7. Chat, refusal and abstention: the DELETE request is refused without any tool call; the question about listening history is answered with "Not enough information".



	A	B	C
Questions	40	40	40
Execution accuracy	73%	87%	100%
Valid SQL rate	93%	100%	100%
Repair success	–	–	–
Schema fact accuracy	100%	100%	100%
Path accuracy	100%	100%	100%
Unsupported claim rate	0%	0%	0%
Abstention accuracy	98%	100%	100%
Mean latency	2.3 s	2.1 s	2.3 s
Input tokens	63,166	130,040	381,786

Figure 8. Management · Benchmark: the A / B / C result table of the last Chinook run.

5. Evaluation

Table 3 shows the benchmark results for both databases (gpt-6-luna, reasoning effort low, one run per configuration, 40 questions each). Execution accuracy and the valid SQL rate are computed over the 15 answerable data questions per database. Figure 9 shows the execution accuracy.

Table 3. Benchmark results per configuration (A: raw DDL, B: docs + graph + JOIN path, C: B + data profile)

Metric	Chinook A	B	C	financial A	B	C
Execution accuracy	73%	87%	100%	67%	80%	80%
Valid SQL rate	93%	100%	100%	73%	80%	87%
Schema fact accuracy	100%	100%	100%	88%	93%	84%
Path accuracy	100%	100%	100%	90%	100%	90%
Unsupported claim rate	0%	0%	0%	0%	0%	0%
Abstention accuracy	98%	100%	100%	88%	88%	90%
Mean latency	2.3 s	2.1 s	2.3 s	2.2 s	2.1 s	2.0 s
Input tokens (40 q.)	63,166	130,040	381,786	40,152	105,962	329,762
Cost (40 q.)	$0.008	$0.015	$0.040	$0.006	$0.012	$0.035

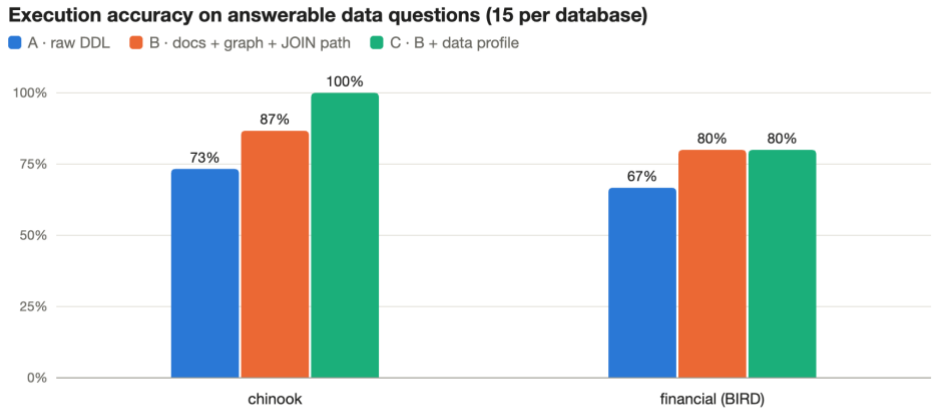


Figure 9. Execution accuracy on the answerable data questions.

What the context adds. On Chinook, schema questions are at the ceiling in every configuration: readable names and declared keys are enough even from the raw DDL. The difference is in the value-dependent questions, as the plan predicted. Without values, A wrote City = 'Montreal' and BillingCity = 'Sao Paulo' (stored as 'Montréal' and 'São Paulo'), 'Science Fiction & Fantasy' (stored as 'Sci Fi & Fantasy'), and abstained on purchased AAC files ('Purchased AAC audio file'). B fixed two of these from the documentation, which had seen the profile's frequent values; C, with the full profile, answered all 15 correctly.

On financial, A loses on Czech codes: without values it cannot know that incoming money is type = 'PRIJEM' or that card withdrawals are operation = 'VYBER KARTOU'. B gets these codes from the generated docs, and on the five value questions B is right on 5, C on 4 and A on 1. Here C does not beat B, because B's documentation already carries the values, and more context can also mislead: C once wrote type = 'VYBER' where card withdrawals are stored with type VYDAJ. For the opaque district columns (A4..A16) the docs honestly say the meaning is unspecified, so B and C abstain where A guesses; BIRD is public, so some of A's hits may come from memorised knowledge rather than reading. A human edit of these descriptions in the Documentation tab would fix them, which is the purpose of the review step.

No configuration ever named a non-existent table or column (0% unsupported claims), and the repair retry was never needed on answerable questions, so repair success has no value. The cost of the richer context is visible: C uses about six times the input tokens of A, but a full 40-question run still costs only $0.04, and latency stays around 2 s per question.

What was implemented. Every component of the plan: parser, profiler with masking and FK inference, graph with path search, LLM documentation with review, the five-tool agent with evidence

panel, the SQL guard, the three-configuration benchmark with all listed metrics, and the three-part UI (pipeline, chat, management with relationships, documentation, databases, masking, settings and benchmark). Deviations from the plan: a hand-written tool loop instead of the Agents SDK (see 3.5); Langfuse instead of Arize Phoenix for tracing; an extra FastAPI layer between core/ and the UI; and the tools are not exposed as an MCP server (an optional extension).

Strengths and weaknesses. The guardrails are enforced in code and covered by tests, the answers are explainable through the tool trace, and the human-in-the-loop management of relationships and descriptions directly improves the context. The limits are:

- Business logic (what an "active customer" is) is not in the schema or the data.
- Valid SQL can still give a wrong answer: the guard checks the shape of a query, not its meaning (the VYBER example above).
- find_path takes a shortest path, which can be wrong in meaning when two routes tie (client to account via district in financial).
- Masking is by column name or user mark; for example client.birth_date in financial stays unmasked until marked.
- The benchmark is small (40 questions per database, one run per configuration), so differences of one or two questions are within noise.

6. References

[1] T. Yu et al., "Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task", EMNLP 2018.

[2] J. Li et al., "Can LLM Already Serve as A Database Interface? A BIg Bench for Large-Scale Database Grounded Text-to-SQLs" (BIRD), NeurIPS 2023 Datasets and Benchmarks. https://bird-bench.github.io/

[3] M. Pourreza, D. Rafiei, "DIN-SQL: Decomposed In-Context Learning of Text-to-SQL with Self-Correction", NeurIPS 2023.

[4] Vanna: open-source RAG framework for text-to-SQL. https://github.com/vanna-ai/vanna

[5] L. Rocha, Chinook Database (SQLite build v1.4.5). https://github.com/lerocha/chinook-database

[6] sqlglot: SQL parser and transpiler. https://github.com/tobymao/sqlglot

[7] NetworkX: network analysis in Python. https://networkx.org/

[8] OpenAI API documentation: Responses API, function calling and structured outputs. https://platform.openai.com/docs

[9] FastAPI. https://fastapi.tiangolo.com/ · Streamlit. https://streamlit.io/ · pyvis. https://pyvis.readthedocs.io/

[10] Langfuse: open-source LLM observability. https://langfuse.com/

Hallgatói nyilatkozatgeneratív mesterséges intelligencia alkalmazásáról

☐ **Nem használtam** semmilyen generatív MI segédeszközt.

☒ **Használtam** generatív MI segédeszközt. Az MI-vel generált tartalmakat ellenőriztem, a generált kimenetek valóságtartalmáról meggyőződtem, az alábbi táblázatban megfelelően jelöltem minden használatot.

Felhasználási módok	**Generatív MI eszköz(ök) neve**	**Érintett részek** (fejezet, oldalszám, hivatkozás)	**Használat becsült aránya** (felhasználási módonként)
Irodalomkutatás	Claude	Related Work, References	70%
Prompt lényegi része	Gather references and releted work for an LLM-based text-to-SQL and schema documentation project.		
Programkód generálása	Claude	Source code	80%
Prompt lényegi része	Implement the planned module (parser, profiler, graph, SQL guard, agent, API routes, Streamlit pages) with tests, following the implementation		
Új ötletek, megoldási javaslatok generálása	Claude	e.g. the FK-candidate rule (name and value match together), the hand-written tool loop instead of the Agents SDK	50%
Prompt lényegi része	Propose a design for this feature: approaches, trade-offs, recommendation.		
Vázlat létrehozása (szövegstruktúra, vázlatpontok)			
Prompt lényegi része			
Szövegblokkok létrehozása			
Prompt lényegi része			
Képek generálása illusztrációs célból	Claude	Architecture diagram	30%
Prompt lényegi része	Create an architecture diagram from the code		
Adatvizualizáció, grafikonok generálása adatpontok alapján			
Prompt lényegi része			
Prezentáció készítése			

Prompt lényegi része			
Egyéb (nevezze meg)			
Prompt lényegi része			
Összesített százalékos érték (a feladat érdemi részére nézve)			40%
Összesített érték rövid, szöveges indoklása: I wrote the task specification, made the design decisions and approved the implementation plans. Most of the implementation, the tests were produced by an AI coding agent under my direction. I ran and checked the code and the benchmark results, and I checked the report's claims against the code and the measured results.			

Figyelem: a fenti százalékértékektől függetlenül az MI-vel generált minden tartalomért a szerző a felelős!